

Guide : Apache Kafka et Déploiement sur Google Cloud Platform

1. Introduction à Apache Kafka : Le Système Nerveux Numérique

Apache Kafka n'est pas une simple base de données, c'est une plateforme de **streaming d'événements** distribuée. Dans une architecture classique, les applications communiquent souvent point à point. Lorsque le nombre d'applications augmente, cela devient ingérable. Kafka résout ce problème en agissant comme un "hub" central où les données circulent sous forme de flux continus. Contrairement à une file d'attente de messages traditionnelle qui supprime le message après lecture, Kafka est un journal de bord immuable : il stocke les événements dans l'ordre où ils arrivent et permet à plusieurs applications de les relire à leur propre rythme.

Pour comprendre Kafka, il faut maîtriser quatre concepts clés. Le **Producer** est l'application (ici ton backend Node.js) qui envoie l'information. Le **Broker** est le serveur Kafka lui-même qui reçoit et stocke les données. Le **Topic** est une catégorie ou un dossier dans lequel les messages sont triés (par exemple web-events). Enfin, le **Consumer** est l'application qui récupère ces données pour les traiter (comme Spark ou un service d'analyse).

2. L'Architecture KRaft : La Modernisation de Kafka

Historiquement, Kafka avait besoin d'un second logiciel nommé ZooKeeper pour gérer l'état du cluster (élection du chef, métadonnées). Les versions récentes utilisent désormais **KRaft** (Kafka Raft). Ce mode simplifie l'architecture en intégrant la gestion des métadonnées directement à l'intérieur de Kafka. Un nœud peut désormais jouer deux rôles : **Broker** (il manipule les données des utilisateurs) et **Controller** (il gère l'intelligence et la cohérence du cluster). C'est cette architecture moderne que nous avons déployée sur Google Cloud.

3. Déploiement sur Google Cloud Platform (GCP)

Le déploiement sur le Cloud introduit des complexités réseaux majeures, notamment la distinction entre les adresses IP internes (utilisées par la machine pour se parler à elle-même) et les adresses IP externes (utilisées par Vercel ou ton PC).

Préparation de l'infrastructure

L'installation commence par la mise en place d'une instance Compute Engine sous Linux. Kafka étant développé en Java et Scala, l'installation d'un environnement d'exécution Java (JRE) est un prérequis indispensable. Après avoir téléchargé les binaires officiels, le défi principal réside dans la configuration du fichier `server.properties`.

Configuration Stratégique du Réseau

Pour qu'un Broker Kafka soit accessible depuis Internet tout en restant stable, nous avons configuré trois paramètres essentiels :

- **listeners** : Définit sur quelles interfaces réseau Kafka écoute. En utilisant 0.0.0.0, on autorise le serveur à accepter des connexions provenant de n'importe quelle source externe.
- **advertised.listeners** : C'est le paramètre le plus critique. C'est l'adresse que Kafka "donne" aux clients (comme Vercel) lors de la première connexion. Si cette adresse est localhost, Vercel essaiera de se connecter à lui-même. Elle doit donc impérativement être l'**IP Publique Statique** de la VM.
- **controller.quorum.voters** : Pour le mode KRaft, ce paramètre définit où se trouve le "cerveau" du cluster. Pour plus de stabilité, nous l'avons fixé sur l'interface locale (127.0.0.1).

4. Mise en œuvre et Commandes Opérationnelles

Initialisation du Stockage

Avant de démarrer Kafka en mode KRaft, il faut générer un identifiant unique pour le cluster et formater le répertoire de stockage des logs. Cette étape crée le fichier meta.properties qui contient l'identité du nœud.

```
# Génération d'un ID et formatage
KAFKA_CLUSTER_ID="fC1Sj7_oTuqLg0AQip-G4g"
bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c
config/kraft/server.properties
```

Gestion du Cycle de Vie

Le démarrage se fait via le script kafka-server-start.sh. Pour vérifier que le serveur est opérationnel, on utilise la commande réseau ss -lntp qui permet de confirmer que le port **9092** (données) et le port **9093** (contrôleur) sont bien en état d'écoute.

Tests de Flux de Données

La validation se fait en deux étapes. D'abord, la création d'un topic :

```
bin/kafka-topics.sh --create --topic web-events --bootstrap-server
localhost:9092
```

Ensuite, l'utilisation d'un **Console Consumer**. C'est un outil de débogage puissant qui affiche en temps réel chaque message JSON envoyé par ton site e-commerce vers le topic.

```
bin/kafka-console-consumer.sh --topic web-events --bootstrap-server
```

```
localhost:9092 --from-beginning
```

5. Sécurité et Firewall Cloud

Une machine sur GCP est protégée par un pare-feu VPC. Même si Kafka est bien configuré, Google bloque par défaut tout trafic entrant sur le port 9092. La création d'une règle de pare-feu avec un **Target Tag** (comme kafka) et une plage source 0.0.0.0/0 est l'étape finale qui permet d'exposer ton infrastructure de données au reste du monde, rendant possible l'intégration de ton backend Node.js déployé sur Vercel.